

TutorBot AI Education Cloud

AWS Cloud Engineering Case Study

B.Tech CSE 2024–2028

ITM Skills University

Name: Rohan Vashisht

Roll No.: 150096724132

Cohort: Jensen Huang

Repository: <https://github.com/RohanVashisht1234/college-AWS>

Project Overview

TutorBot AI Education Cloud is a cloud-native educational management platform designed to demonstrate enterprise cloud engineering concepts using AWS.

The project addresses the challenge of managing educational operations through a centralized, scalable, secure, and highly available cloud platform.

The implementation focuses on:

- Cloud Infrastructure Design
- Linux Administration
- Docker Containerization
- AWS Networking

- Database Management
- Monitoring & Logging
- Automation
- Cost Optimization
- Operational Dashboards
- Role-Based Access Control

Primary Objective: The goal is not application complexity, but rather demonstrating practical cloud computing concepts and AWS service integration.

Problem Statement

TutorBot AI Education Cloud is experiencing rapid growth across multiple operational regions. Existing operations rely on disconnected systems, spreadsheets, manual workflows, and isolated reporting environments.

The organization requires:

- Centralized operational management
- Secure access control
- Reporting and analytics
- Monitoring and observability
- Disaster recovery readiness
- Scalability for future growth
- High availability architecture
- Cloud-native deployment practices

Solution Architecture

The platform was designed using AWS services following modern cloud deployment best practices.

Architecture Components

Edge Layer

- CloudFront
- Amazon S3

Compute Layer

- Application Load Balancer
- EC2 Instance 1
- EC2 Instance 2

Application Layer

- FastAPI Backend
- Docker Containers

Data Layer

- Amazon RDS MySQL

Monitoring Layer

- Amazon CloudWatch
- CloudWatch Dashboards
- CloudWatch Alarms

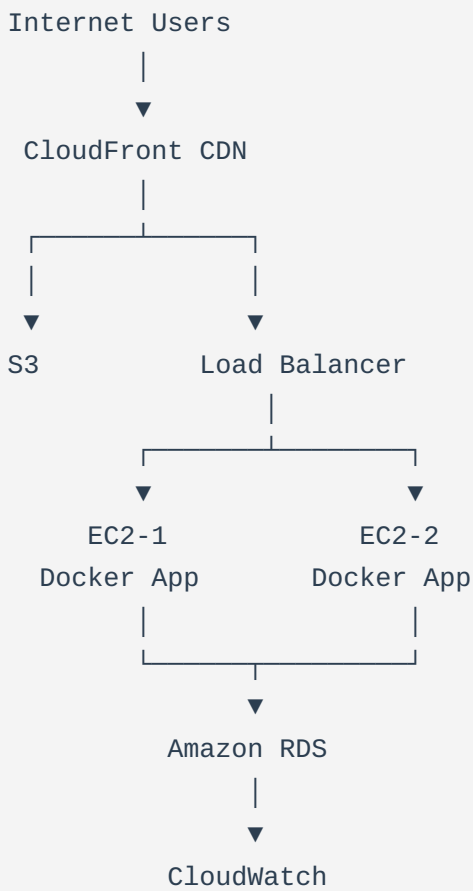
Security Layer

- IAM Roles
- Security Groups
- AWS Systems Manager

Container Registry

- Amazon ECR

High Level Architecture



AWS Infrastructure Implemented

Networking

VPC

Dedicated Virtual Private Cloud created for TutorBot.

Subnets

- Public Subnet A
- Public Subnet B

Internet Gateway

Provides internet connectivity.

Route Tables

Configured for public subnet routing.

Security Groups

Application Security Group:

- HTTP 80
- SSH / SSM Access

Database Security Group:

- MySQL 3306
- Restricted to application servers

Compute Infrastructure

EC2 Instance 1

Purpose:

- FastAPI Application
- Docker Runtime

Responsibilities:

- Serve API Requests
- Process Application Logic

EC2 Instance 2

Purpose:

- High Availability
- Load Balancer Target

Responsibilities:

- Serve API Requests
- Redundancy

Load Balancer

Application Load Balancer configured to:

- Distribute traffic
- Improve availability
- Support scaling
- Route requests to healthy instances

Health Checks: `/health`

Database Layer

Amazon RDS MySQL

Used for:

- Student Records
- Courses
- Assignments
- Enrollments
- Authentication Data

Advantages:

- Managed Backups
- Automated Maintenance
- High Availability Ready
- Monitoring Integration

Backend Technology Stack

FastAPI

Features: REST API, Authentication, Role-Based Access, Dashboard Metrics, Reporting APIs

Database ORM

SQLAlchemy

Validation

Pydantic

Environment Management

Python Dotenv

Backend Features

Authentication

Mock Login System

Roles: Teacher, Student

Credentials:

Username: user
Password: 1234

Teacher Capabilities: Manage Students, Manage Courses, Create Assignments, Generate Reports

Student Capabilities: View Courses, View Assignments, Access Dashboard

API Modules

- **Authentication:** Login, Current User
- **Students:** Create, Read, Update, Delete
- **Courses:** Create, Read, Update, Delete
- **Assignments:** Create, Read
- **Enrollments:** Create, Read
- **Reports:** Summary Reports
- **Monitoring:** Health Check, Metrics, Dashboard Statistics

Frontend Technology Stack

- **React:** Single Page Application
- **TypeScript:** Type Safe Development
- **Vite:** Fast Build Tool
- **Recharts:** Analytics Visualizations
- **React Router:** Routing

Frontend Features

- **Login Portal:** Role Selection (Teacher, Student)
- **Dashboard:** Displays Students, Courses, Enrollments, Assignments
- **Student Management:** Add, Edit, Delete Students
- **Course Management:** Add, Edit, Delete Courses
- **Assignment Management:** Create, View Assignments
- **Reporting Dashboard:** KPI Overview, Operational Metrics, System Statistics
- **Cloud Health Dashboard:** Displays Application Status, Database Status, Monitoring Information

Docker Implementation

Backend Container

FastAPI application packaged using Docker.

Benefits:

- Portability
- Consistency
- Simplified Deployment

Container Registry

Amazon ECR

Used for:

- Storing Docker Images
- Version Management
- Deployment Source

Monitoring & Observability

Amazon CloudWatch

Implemented:

- **Metrics:** CPU Utilization, Network Usage, Memory Monitoring, Application Health
- **Dashboards:** Infrastructure Metrics, Resource Utilization, Operational Visibility
- **Alarms:** Threshold Monitoring, Performance Alerts

Linux Administration

Implemented Concepts:

- **User Management:** IAM Integration, Systems Manager Access
- **Process Monitoring:** Docker Containers, Application Processes
- **Log Monitoring:** Application Logs, System Logs
- **Service Management:** Docker Runtime, EC2 Services

Automation

Backup Automation

Automated RDS Snapshot Script.

- **Purpose:** Disaster Recovery, Data Protection
- **Features:** Timestamped Snapshots, Automated Scheduling

Cron Jobs

Implemented for Backup Scheduling and Maintenance Tasks.

Security Implementation

IAM

Configured for EC2 Roles, ECR Access, CloudWatch Access, Systems Manager Access.

Security Groups

Application Security (Controlled HTTP Access), Database Security (Restricted MySQL Access).

Systems Manager

Secure Administration Without SSH Exposure.

Benefits: Improved Security, Centralized Management

Disaster Recovery Strategy

Database

Amazon RDS Snapshots. **Recovery Process:**

1. Create Snapshot
2. Restore Database
3. Reconnect Application

Application

Docker Images Stored In ECR. **Recovery Process:**

1. Launch New EC2
2. Pull Container
3. Start Application

Scalability Design

Current Architecture:

- 2 EC2 Instances
- Load Balancer
- Managed Database

Future Expansion:

- Auto Scaling Groups
- Multi-AZ Database
- Multi-Region Deployment

- WAF Integration
- CI/CD Pipelines

Cost Optimization

Implemented Strategies:

- **Compute:** Right-sized EC2 instances
- **Database:** Managed RDS
- **Storage:** S3 Static Hosting
- **CDN:** CloudFront Caching
- **Monitoring:** Essential Metrics Only
- **Containers:** Lightweight Docker Images

Estimated Monthly Cost

Service	Estimated Cost (USD/Month)
2 EC2 Instances	\$16
Application Load Balancer	\$18
Amazon RDS MySQL	\$15
Amazon S3	\$2
CloudFront	\$2
Amazon ECR	\$1
CloudWatch	\$3
Data Transfer	\$3
Total	~\$60

Approximate Monthly Cost: \$55 – \$65 USD/month (Educational workloads and low traffic usage).

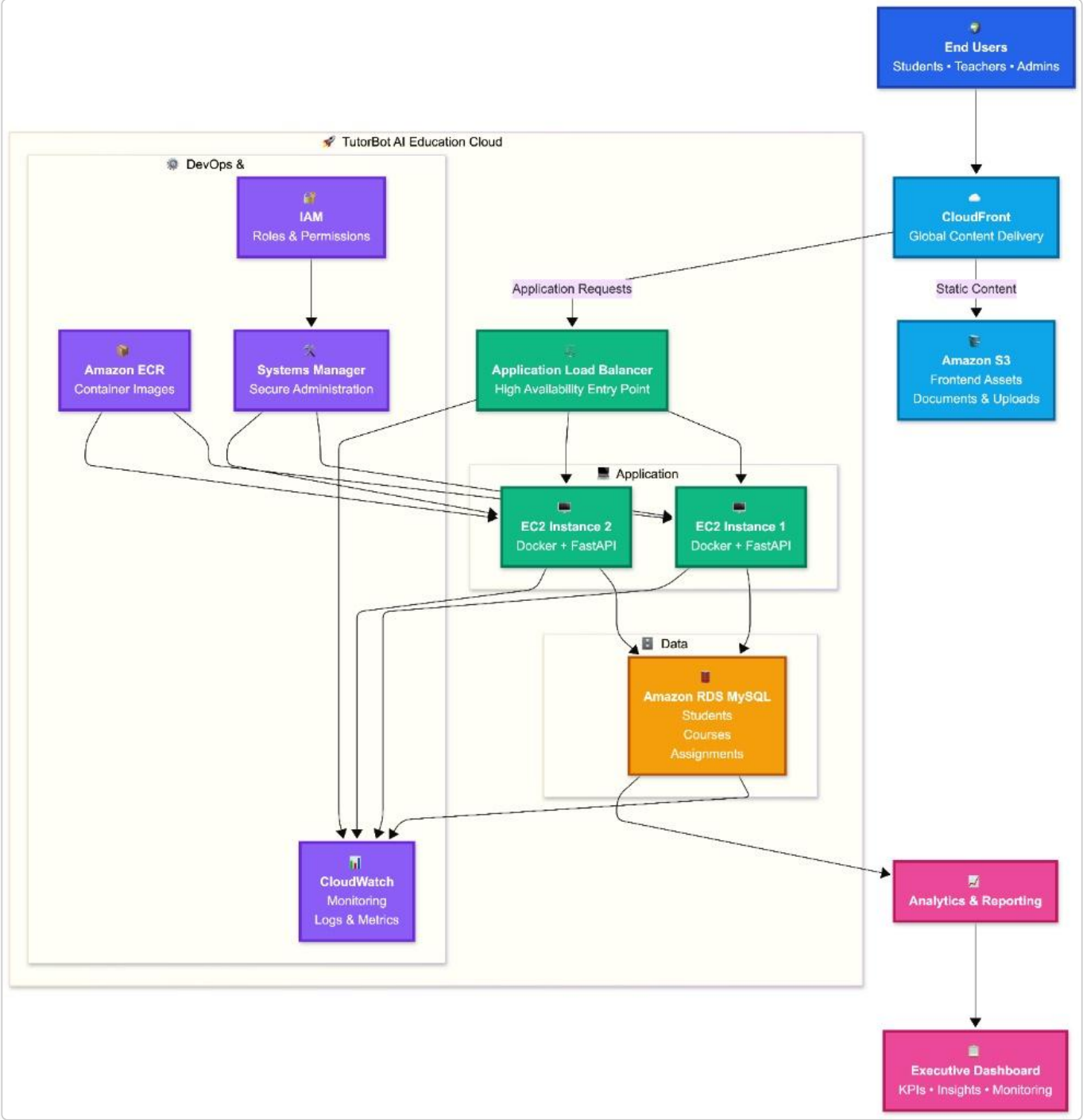
Repository Structure

```
.
├── README.md
├── backend
│   ├── Dockerfile
│   ├── README.md
│   ├── __pycache__
│   │   └── main.cpython-314.pyc
│   ├── app
│   │   ├── __pycache__
│   │   │   └── main.cpython-314.pyc
│   │   └── main.py
│   ├── pyproject.toml
│   └── uv.lock
├── frontend
│   ├── README.md
│   ├── deno.lock
│   ├── index.html
│   ├── node_modules
│   ├── package.json
│   ├── src
│   │   ├── App.tsx
│   │   ├── api.ts
│   │   ├── auth.tsx
│   │   ├── components
│   │   │   ├── AppShell.tsx
│   │   │   ├── ConfirmDialog.tsx
│   │   │   ├── Header.tsx
│   │   │   ├── Modal.tsx
│   │   │   ├── Sidebar.tsx
│   │   │   └── StatCard.tsx
│   │   ├── main.tsx
│   │   ├── pages
│   │   │   ├── AssignmentsPage.tsx
│   │   │   ├── CloudHealthPage.tsx
│   │   │   ├── CoursesPage.tsx
│   │   │   ├── DashboardPage.tsx
│   │   │   ├── LoginPage.tsx
│   │   │   ├── ReportsPage.tsx
│   │   │   ├── StudentsPage.tsx
│   │   │   └── SystemDesignPage.tsx
│   │   ├── styles.css
│   │   ├── types.ts
│   │   └── vite-env.d.ts
│   └── tsconfig.json
```

```
|   ├── tsconfig.tsbuildinfo
|   └── vite.config.ts
└── screenshots
    ├── archietcture_diagram.png
    ├── cloudwatch_dashboard.png
    ├── cron.png
    ├── ec2_instance_1.png
    ├── ec2_instance_2.png
    ├── ecr.png
    ├── load_balancer.png
    ├── role.png
    ├── target_group.png
    └── vpc_and_subnets.png
```


Screenshots

Architecture Diagram



VPC and Subnets

The screenshot displays the AWS VPC console interface. The left sidebar shows the navigation menu with categories like Virtual private cloud, Security, and PrivateLink and Lattice. The main content area is titled 'vpc-074f0e582176be1cc' and contains a 'Details' tab. The details are organized into four columns: VPC ID (vpc-074f0e582176be1cc), State (Available), Block Public Access (Off), and DNS hostnames (Enabled). Other details include DNS resolution (Enabled), Main network ACL (acl-04a9e6f4c4bb52968), IPv6 CIDR (Network border group), Encryption control ID, Tenancy (default), Default VPC (No), Network Address Usage metrics (Disabled), Encryption control mode, DHCP option set (dopt-0581f9fd704633309), IPv4 CIDR (10.0.0.0/16), Route S3 Resolver DNS Firewall rule groups, Main route table (rtb-02163617909b932fc), and IPv6 pool. Below the details is a 'Resource map' tab showing a diagram of the VPC resources: VPC, Subnets (2), Route tables (2), and Network Connections (1). The VPC is connected to two subnets (ap-south-1a and ap-south-1b) and two route tables (rtb-02163617909b932fc and rtb-037378ae86be2331a). The subnets are connected to a network connection (igw-068aae142a86c2726).

EC2 Instance 1

The screenshot displays the AWS EC2 console interface. The left sidebar shows the navigation menu with categories like EC2, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area is titled 'Instance summary for i-0bd6276fd43eb6e00 (APP-Instance-1)' and contains a 'Details' tab. The details are organized into four columns: Instance ID (i-0bd6276fd43eb6e00), Public IPv4 address (43.205.136.85), Private IPv4 addresses (10.0.1.59), and Public DNS (ec2-43-205-136-85.ap-south-1.compute.amazonaws.com). Other details include Instance state (Running), Private IP DNS name (ip-10-0-1-59.ap-south-1.compute.internal), Instance type (t3.micro), VPC ID (vpc-074f0e582176be1cc), Subnet ID (subnet-0080edc719e54df04), Instance ARN (arn:aws:ec2:ap-south-1:575084400813:instance/i-0bd6276fd43eb6e00), IAM role (AWS-ec2-role), IMDSv2 (Required), Operator, and AWS Compute Optimizer finding (Opt-in to AWS Compute Optimizer for recommendations). Below the details is a 'Details' tab showing instance details: AMI ID (ami-0e38835daf6b8a2b9), AMI name (al2023-ami-2023.12.20260611.0-kernel-6.1-x86_64), Stop protection (Disabled), Monitoring (disabled), Allowed image, Launch time (Sat Jun 20 2026 21:17:39 GMT+0530 (India Standard Time) (1 day)), Platform details (Linux/UNIX), Termination protection (Disabled), and AMI location (amazon/al2023-ami-2023.12.20260611.0-kernel-6.1-x86_64).

EC2 Instance 2

EC2

Dashboard

AWS Global View

Events

Instances

Instances Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target Groups

Trust Stores

Auto Scaling

Auto Scaling Groups

Instance summary for i-0bd6276fd43eb6e00 (APP-Instance-1)

Updated less than a minute ago

Instance ID

i-0bd6276fd43eb6e00

IPv6 address

-

Hostname type

IP name: ip-10-0-1-59.ap-south-1.compute.internal

Answer private resource DNS name

-

Auto-assigned IP address

43.205.136.85 [Public IP]

IAM role

AWS-exam-ec2-role

IMDSv2

Required

Operator

-

Public IPv4 address

43.205.136.85 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-10-0-1-59.ap-south-1.compute.internal

Instance type

t3.micro

VPC ID

vpc-074f0e582176be1cc

Subnet ID

subnet-0080edc719e54df04

Instance ARN

arn:aws:ec2:ap-south-1:575084400813:instance/i-0bd6276fd43eb6e00

Private IPv4 addresses

10.0.1.59

Public DNS

ec2-43-205-136-85.ap-south-1.compute.amazonaws.com | open address

Elastic IP addresses

-

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name

-

Managed

false

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

Instance details

AMI ID

ami-0e38835daf6b8a2b9

AMI name

al2023-ami-2023.12.20260611.0-kernel-6.1-x86_64

Stop protection

Disabled

Monitoring

disabled

Allowed image

-

Launch time

Sat Jun 20 2026 21:17:39 GMT+0530 (India Standard Time) (1 day)

Platform details

Linux/UNIX

Termination protection

Disabled

AMI location

amazon/al2023-ami-2023.12.20260611.0-kernel-6.1-x86_64

Load Balancer

EC2

Dashboard

AWS Global View

Events

Instances

Instances Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target Groups

Trust Stores

Auto Scaling

Auto Scaling Groups

hello-alb

Details

Load balancer type

Application

Scheme

Internet-facing

Status

Active

Hosted zone

ZP97RAFLXTNZK

VPC

vpc-074f0e582176be1cc

Availability Zones

subnet-0080edc719e54df04 ap-south-1a (aps1-az1)

subnet-08e3775776be24d ap-south-1b (aps1-az3)

Load balancer IP address type

IPv4

Date created

June 20, 2026, 02:09 (UTC+05:30)

Load balancer ARN

arn:aws:elasticloadbalancing:ap-south-1:575084400813:loadbalancer/app/hello-alb/e8ba8da0d4e58394

DNS name

hello-alb-625156640.ap-south-1.elb.amazonaws.com (A Record)

Listeners and rules (1)

Manage rules

Manage listener

Add listener

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.

Filter listeners

Protocol/Port

Default action

Rules

ARN

Security policy

Default SSL/TLS certificate

mTLS

HTTP:80

Forward to target group

hello-tg: 1 (100%)

Target group stickiness: Off

1 rule

ARN

Not applicable

Not applicable

Not applicable

Target Group

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

Load Balancers

Target Groups

Trust Stores

Auto Scaling

hello-tg

Details

arn:aws:elasticloadbalancing:ap-south-1:575084400813:targetgroup/hello-tg/f6519f9595dad076

Target type

Instance

IP address type

IPv4

Protocol : Port

HTTP: 80

Load balancer

hello-alb

Protocol version

HTTP1

VPC

vpc-074f0e582176be1cc

2

Total targets

2

Healthy

0

Unhealthy

0

Unused

0

Initial

0

Draining

0

Anomalous

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (2)

Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

Instance ID	Name	Port	Zone	Health status	Health status details	Admini...	Overri...	Launch...
i-0bd6276fd43eb6e00	APP-Instance-1	80	ap-south-1a (a...	Healthy	-	No override.	No overri...	June 20, 2...
i-0c657e14e2a97a88b	APP-Instance-2	80	ap-south-1b (a...	Healthy	-	No override.	No overri...	June 20, 2...

Amazon ECR

Amazon ECR

Private registry

Repositories

Features & Settings

Public registry

Repositories

Settings

ECR public gallery

Amazon ECS

Amazon EKS

Getting started

Documentation

Private repositories (1)

View push commands

Delete

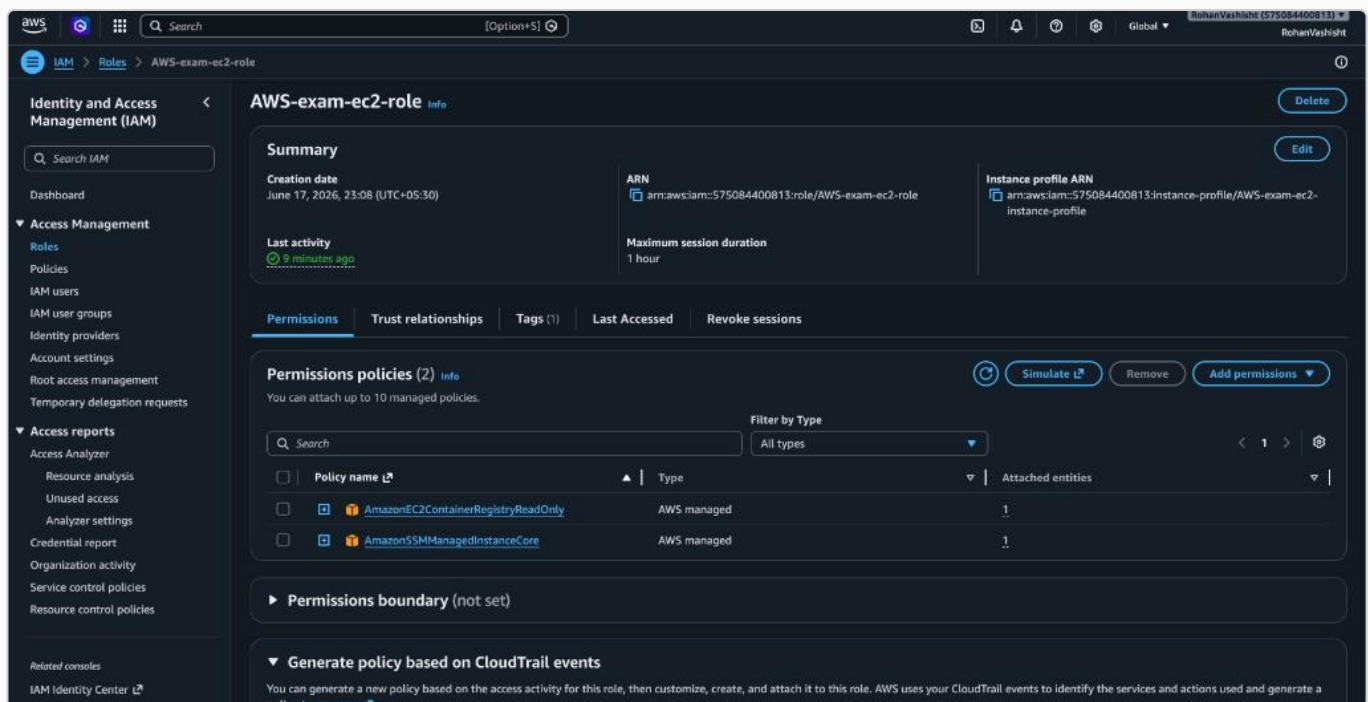
Actions

Create repository

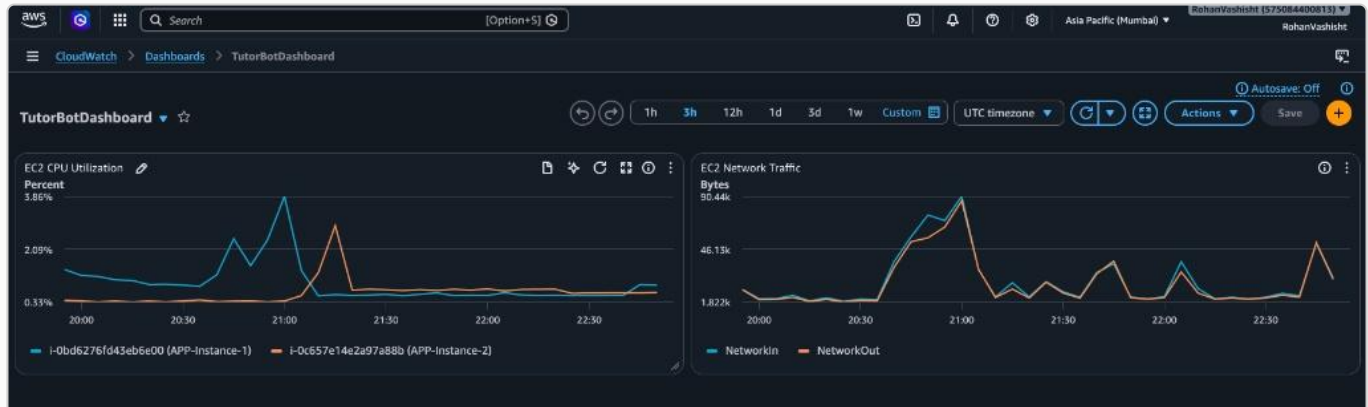
Search by repository substring

Repository name	URI	Created at	Tag immutability	Encryption type
fastapi-demo	575084400813.dkr.ecr.ap-sout h-1.amazonaws.com/fastapi-demo	June 20, 2026, 22:43:21 (UTC+05.5)	Mutable	AES-256

IAM Role



CloudWatch Dashboard



Automation Cron Job

```
Session ID: root-7avi7rbxpf9tvxagobvz9ale9a Shortcuts Instance ID: i-Obd6276fd43eb6e00 (APP-Instance-1)

sh-5.2$ vi backup-rds.sh
sh-5.2$ vi backup-rds.sh
sh-5.2$ sudo vi backup-rds.sh
sh-5.2$ chmod +x backup-rds.sh
chmod: changing permissions of 'backup-rds.sh': Operation not permitted
sh-5.2$ sudo chmod +x backup-rds.sh
sh-5.2$ ./backup-rds.sh
Creating snapshot: demo-mysql-snapshot-20260621-224852
{
  "DBSnapshot": {
    "DBSnapshotIdentifier": "demo-mysql-snapshot-20260621-224852",
    "DBInstanceIdentifier": "demo-mysql",
    "Engine": "mysql",
    "AllocatedStorage": 20,
    "Status": "creating",
    "Port": 3306,
    "AvailabilityZone": "ap-south-1b",
    "VpcId": "vpc-074f0e582176belcc",
    "InstanceCreateTime": "2026-06-21T00:20:12.983000+00:00",
    "MasterUsername": "admin",
    "EngineVersion": "8.0.45",
    "LicenseModel": "general-public-license",
    "SnapshotType": "manual",
    "Iops": 3000,
    "StorageThroughput": 125,
    "OptionGroupName": "default:mysql-8-0",
    "PercentProgress": 0,
    "StorageType": "gp3",
    "Encrypted": false,
    "DBSnapshotArn": "arn:aws:rds:ap-south-1:575084400813:snapshot:demo-mysql-snapshot-20260621-224852",
    "IAMDatabaseAuthenticationEnabled": false,
    "ProcessorFeatures": [],
    "DbiResourceId": "db-7JEDVQWMTJIOTSDZLM5T4NX6Y",
    "TagList": [],
    "SnapshotTarget": "region",
    "DedicatedLogVolume": false
  }
}
Snapshot creation initiated.
sh-5.2$ crontab -e
crontab: installing new crontab
sh-5.2$
```

Learning Outcomes

This project demonstrates:

- AWS Infrastructure Design
- VPC Networking
- EC2 Administration
- Load Balancing
- Docker Containerization
- Amazon ECR
- Amazon RDS

- Amazon S3
- CloudFront
- IAM Security
- CloudWatch Monitoring
- Linux Administration
- Shell Automation
- Cost Optimization
- Disaster Recovery Planning
- Full Stack Cloud Deployment